# Healthcare System Design Decisions - Meeting Brief

## Purpose

This document summarizes all critical design decisions made in D2 for discussion with the professor. Each decision includes the rationale and alternatives considered.

---

## 1. DATABASE DESIGN DECISIONS

### 1.1 ISA Inheritance (User → Patient/Doctor)

**Decision:** Use ISA inheritance where User is the base entity, with Patient and Doctor extending it through one-to-one relationships.

**Rationale:**

- Patients and doctors have different profile data requirements
- Patient needs: date_of_birth, allergies, emergency contacts, medical_history
- Doctor needs: license_number, specialization, years_experience
- Storing all fields in a single User table would create many NULL values
- Violates database normalization (3NF)
- Maintains clean separation between authentication and profile data

**Alternative Considered:** Single User table with all patient and doctor fields

- **Rejected because:** Poor data integrity, excessive NULL values, harder to maintain

**If Professor Asks:** "Why not just use Django's built-in User groups?"

- **Answer:** Groups handle permissions, not data storage. We need different profile fields for patients vs doctors, which requires separate tables.

---

### 1.2 No Separate Admin Entity

**Decision:** Admins exist only in User table with `user_type='admin'`, no separate Admin table.

**Rationale:**

- D1 admin requirements are actions/permissions (user management, system monitoring), not data storage
- Admin needs no additional profile data beyond User fields (email, name)
- Creating empty or near-empty tables violates database design principles
- Follows rule: "Create tables only when additional data exists"

**If Professor Asks:** "What if admins need more data later?"

- **Answer:** We can add an Admin entity in future sprints if requirements change. For MVP, YAGNI principle applies.

---

## 1.3 Separate Prescription vs Medication Tables

**Decision:** Keep Prescription (historical) and Medication (current tracking) as separate entities.

**Rationale:**

- **Prescription:** Immutable historical record of what doctor ordered during appointment
- **Medication:** Mutable current state of what patient is taking
- Different lifecycles:
  - Patient stops medication early → `Medication.is_active = false`
  - Original Prescription remains unchanged (audit trail)
- Supports D1 requirement: "Medication tracking and reminders"
- Patients can add non-prescribed medications (vitamins, supplements) → `prescription_id` is nullable

**Example Workflow:**

1. Doctor prescribes Metformin → Prescription created (immutable)
2. System creates Medication from Prescription → `is_active = true`
3. Patient stops taking it → `Medication.is_active = false`
4. Prescription remains unchanged

**Alternative Considered:** Single Medication table combining both

- **Rejected because:** Loses historical accuracy, can't distinguish prescribed vs non-prescribed medications

**If Professor Asks:** "Isn't this over-engineering?"

- **Answer:** No, these are distinct business concepts with different use cases. Audit trails require immutable prescriptions, while medication tracking requires mutable state.

---

## 1.4 Separate MedicalRecord vs MedicalDocument

**Decision:** MedicalRecord stores text notes, MedicalDocument stores binary files.

**Rationale:**

- **MedicalRecord:** Structured text from doctor visits (diagnosis, symptoms, treatment)
- **MedicalDocument:** Binary files (X-rays, PDFs, lab reports)
- Different data types require different storage (database vs file system)
- Different relationships:
  - MedicalRecord: 1:1 with Appointment
  - MedicalDocument: 1:N with Patient (not always tied to appointments)
- Real-world scenario: Patient uploads previous records from another hospital → no appointment in our system

**Alternative Considered:** Single table for all medical data

- **Rejected because:** Mixing text and binary is inefficient, different use cases

---

## 1.5 PostgreSQL-Specific Features

**Decision:** Use PostgreSQL's `array` type for allergies and `JSONB` type for medical_history.

**Rationale:**

- **Array for allergies:** `["penicillin", "shellfish", "peanuts"]`
  - Native list storage without junction table
  - Efficient querying: `WHERE 'penicillin' = ANY(allergies)`
  - Simpler schema
- **JSONB for medical_history:**

```json
{
  "conditions": ["diabetes type 2"],
  "surgeries": [{"type": "appendectomy", "date": "2020-05-15"}],
  "family_history": {"diabetes": true}
}
```

- Flexible schema for varying patient data
- No rigid structure needed for medical history
- Can query within JSON: `WHERE medical_history->'conditions' @> '["diabetes"]'`

**Alternative Considered:** Separate normalized tables (Allergy, MedicalCondition, Surgery, etc.)

- **Rejected because:** Over-engineering for student project scope, unnecessary complexity

**If Professor Asks:** "What about portability to MySQL?"

- **Answer:** D1 specified PostgreSQL. These features align with our chosen tech stack and project requirements.

---

## 1.6 Unique Constraint on Appointments

**Decision:** `UNIQUE(doctor_id, appointment_date)` constraint on Appointments table.

**Rationale:**

- Prevents double-booking of doctor time slots
- Database-level enforcement (more reliable than application-level checks)
- Returns `409 Conflict` error if violated
- Business rule: One doctor cannot have two appointments at same time

---

## 1.7 No AuditLog Entity

**Decision:** No AuditLog table in initial design.

**Rationale:**

- Not explicitly required in D1
- D1 admin "system monitoring" interpreted as basic statistics, not audit trails
- Can be added as future enhancement (mentioned in Security section)
- Keeps initial MVP scope manageable

**If Professor Asks:** "What about compliance/security?"

- **Answer:** We identified audit logging as important for production systems and documented it as a future enhancement. For the student project MVP, we prioritized core functionality.

---

# 2. SYSTEM ARCHITECTURE DECISIONS

## 2.1 Three-Tier Architecture

**Decision:** Separate presentation (React), application (Django), and data (PostgreSQL) tiers.

**Rationale:**

- Clear separation of concerns
- Enables parallel development (frontend/backend teams work independently)
- Each tier can be updated independently
- Simplifies testing (can test backend without frontend)
- Industry standard pattern

**If Professor Asks:** "Why not a monolithic Django app with templates?"

- **Answer:** D1 specified React frontend. Three-tier architecture is necessary for separate frontend/backend and matches our tech stack requirements.

---

## 2.2 RESTful API Design

**Decision:** All frontend-backend communication via REST API using JSON.

**Rationale:**

- Stateless communication (scalable)
- Standard HTTP methods (GET, POST, PATCH, DELETE)
- Clear separation between frontend and backend
- Easy to test endpoints independently (Postman, curl)
- Matches Django REST Framework strengths

---

## 2.3 Token-Based Authentication

**Decision:** Use token authentication, not session-based.

**Rationale:**

- **Stateless:** Server doesn't store session data
- **Scalable:** Works across multiple servers
- **Perfect for separate frontend/backend:** React stores token, includes in requests
- Django REST Framework provides built-in TokenAuthentication
- Simpler than sessions for API-based architecture

**Alternative Considered:** Session-based authentication

- **Rejected because:** Doesn't work well with separate frontend/backend, harder to scale

**If Professor Asks:** "What about JWT?"

- **Answer:** Django's simple token auth is sufficient for MVP. We documented JWT with expiration as a future enhancement for production (mentioned in Security section).

---

# 3. API DESIGN DECISIONS

## 3.1 RESTful Endpoint Structure

**Decision:** Follow REST conventions for URL patterns and HTTP methods.

**Examples:**

- `GET /api/patients/` - List all patients
- `GET /api/patients/5/` - Get patient 5
- `POST /api/appointments/` - Create appointment
- `PATCH /api/appointments/10/` - Update appointment 10

**Rationale:**

- Industry standard
- Predictable patterns
- Clear semantics (HTTP method indicates action)

---

## 3.2 Nested Routes for Related Resources

**Decision:** Use nested routes for related resources.

**Examples:**

- `GET /api/patients/5/medical-records/` - Get patient 5's records
- `GET /api/patients/5/prescriptions/` - Get patient 5's prescriptions

**Rationale:**

- Clearly shows relationship in URL
- Intuitive for frontend developers
- Matches resource hierarchy

---

## 3.3 Query Parameters for Filtering

**Decision:** Use query parameters for filtering, not separate endpoints.

**Examples:**

- `GET /api/appointments/?status=scheduled`
- `GET /api/doctors/?specialization=cardiology`

**Rationale:**

- Flexible filtering without endpoint proliferation
- Standard REST practice

- Easy to combine multiple filters

---

# 4. SECURITY DECISIONS

## 4.1 Role-Based Access Control (RBAC)

**Decision:** Use `user_type` field with custom permission classes.

**Permission Classes:**

- `IsPatient`, `IsDoctor`, `IsAdmin` - Role-based
- `IsOwnerOrDoctor` - Object-level (patients see own data, doctors see patient data)

**Rationale:**

- Matches D1 three user types
- Granular control (both role-level and object-level)
- Enforced at "reception desk" (view level) as discussed

**If Professor Asks:** "Why not use Django's built-in Groups?"

- **Answer:** Groups are useful but our permission logic is simpler with direct `user_type` checks. We need object-level permissions (e.g., patient can only see OWN records), which requires custom classes anyway.

---

## 4.2 Two Main Vulnerabilities Addressed

**Decision:** Focus on SQL Injection and Unauthorized Access as the two required vulnerabilities.

**Rationale:**

- Capstone guidelines require "at least 2 known vulnerabilities"
- SQL Injection: Django ORM handles automatically (parameterized queries)
- Unauthorized Access: Most relevant for healthcare data (permission classes, object-level checks)
- Also implemented CSRF and XSS protection (Django/React defaults)

**If Professor Asks:** "Why not focus on XSS or CSRF?"

- **Answer:** Django and React handle those automatically through output escaping and CSRF middleware. We documented them but focused detail on SQL injection and unauthorized access since those require our explicit design decisions.

---

## 4.3 Password Hashing

**Decision:** Use Django's default PBKDF2 algorithm.

**Rationale:**

- Industry standard
- Django default (260,000 iterations)
- No need to implement custom hashing
- Secure and well-tested

---

# 5. UI/UX DECISIONS

## 5.1 Six Core Screens

**Decision:** Landing page, login, registration, patient dashboard, doctor dashboard, admin dashboard.

**Rationale:**

- Covers all three user types
- Demonstrates complete user flow
- Sufficient for D2 design phase
- Quality over quantity (6 polished screens vs 20 mediocre ones)

**If Professor Asks:** "Where are the other screens (appointment booking, medical records, etc.)?"

- **Answer:** D2 is the design phase. We focused on core screens for each user type. Detailed screens for all features will be built during D3/D4 implementation based on these design patterns.

---

## 5.2 Figma for Mockups

**Decision:** Use Figma for all UI/UX mockups.

**Rationale:**

- Industry-standard design tool
- Easy collaboration (shareable link)
- Export to PNG for documentation
- Team member (Shahrukh) familiar with it

---

# 6. SCOPE DECISIONS (What We Didn't Include)

## 6.1 No Hospital Infrastructure

**Decision:** System manages appointments and records, NOT hospital resources (rooms, beds, departments).

**Rationale:**

- D1 interpreted "hospital/clinic management" as user/appointment management
- Hospital infrastructure is out of scope for student project
- Focus on core healthcare interactions (patient-doctor-admin)

**If Professor Asks:** "Your guidelines mention hospital/clinic management. Where is that?"

- **Answer:** We interpreted this as managing the clinic's appointment system and user base, which is what admins do in our system. Full hospital resource management (beds, rooms, departments) would be a separate, much larger system.

---

## 6.2 No Telemedicine

**Decision:** Excluded telemedicine video consultations from additional features.

**Rationale:**

- D1 chose "Medical imaging upload" and "Medication tracking" as the 2 additional features
- Telemedicine requires complex WebRTC integration
- Beyond scope for timeline
- Imaging and medication tracking are more achievable

---

### 6.3 No Real-Time Features

**Decision:** No WebSocket/real-time updates in initial design.

**Rationale:**

- REST API is sufficient for all requirements
- Real-time updates (appointment notifications) can be added as enhancement
- Polling or page refresh acceptable for MVP

---

## 7. JUSTIFICATION SUMMARY FOR PROFESSOR

### Why Django?

- D1 tech stack decision
- Built-in authentication, ORM, security features
- Django REST Framework for APIs
- Python familiarity from algorithms course

### Why PostgreSQL?

- D1 tech stack decision
- Advanced features (JSONB, arrays) useful for medical data
- Reliable, industry-standard
- Better for complex relationships than MySQL

### Why React?

- D1 tech stack decision
- Component-based matches three user interfaces
- Team member experience
- Good for single-page applications

### Why This Database Schema?

- Supports all D1 functional requirements
- Follows 3NF normalization
- Balances flexibility (JSON fields) with structure (normalized tables)
- PostgreSQL-specific features reduce complexity

### Why These Security Measures?

- Meets capstone requirement (2+ vulnerabilities addressed)
- Django/DRF handle most security automatically
- Permission classes implement business rules from D1
- Realistic for student project timeline

# 8. POTENTIAL TOUGH QUESTIONS & ANSWERS

### Q: "Why didn't you use Django's built-in User model?"

**A:** We did! We extended AbstractUser to add `user_type` field. This gives us all Django's auth features plus our role distinction.

### Q: "Your API endpoints look simple. Where's the complexity?"

**A:** Complexity is in the permission system and business logic, not URL structure. Simple, RESTful endpoints are a design goal, not a weakness.

### Q: "Why no microservices?"

**A:** Microservices add deployment complexity unnecessary for a student project. Monolithic Django backend is appropriate for our scale and timeline.

### Q: "What about HIPAA compliance?"

**A:** We implemented security fundamentals (encryption, authentication, authorization). Full HIPAA compliance involves legal/organizational policies beyond technical implementation and outside student project scope.

### Q: "Why separate Prescription and Medication? Seems redundant."

**A:** They serve different purposes. Prescription = immutable audit trail (what doctor ordered). Medication = mutable tracking (what patient is taking). This separation is necessary for our medication tracking feature requirement.

### Q: "Your SQL schemas seem detailed for a design document."

**A:** We included them to show we understand database implementation and constraints. They document our design decisions (unique constraints, cascading deletes, indexes) which are design concerns, not just implementation details.

### Q: "Only 6 UI screens? That's not complete."

**A:** D2 is design phase, not implementation. Six screens demonstrate design patterns for all three user types. Complete implementation happens in D3/D4. Quality over quantity for design mockups.

### Q: "Why no deployment architecture?"

**A:** Deployment is D5 (weeks 10-12). D2 focuses on application design. We'll use Docker as specified in capstone guidelines when we reach deployment phase.

---

# 9. KEY POINTS TO EMPHASIZE

1. **Every decision traces back to D1 requirements** - We didn't add unnecessary features
2. **We followed normalization principles** - Database design is clean and follows 3NF
3. **Security is built-in, not bolted-on** - Permission checks at every level
4. **Design balances complexity and practicality** - Sophisticated enough to be realistic, simple enough to complete
5. **We documented alternatives considered** - Shows thoughtful decision-making

6. **Tech stack justified in D1** - D2 implements that tech stack appropriately
7. **Scope was carefully managed** - No scope creep, focused on core requirements

---

# 10. WHAT TO BRING TO MEETING

- [x] This brief (printed)
- [x] D1 document (for reference)
- [x] D2 document (submitted version)
- [x] Figma link ready to show
- [x] Any clarifying questions for professor

---

# 11. TEAM MEMBER RESPONSIBILITIES

**Emranul (Backend Developer/You):**

- Database design decisions
- API specification
- Security architecture
- All technical backend rationale

**Shahrukh (Frontend Developer):**

- UI/UX mockups
- Figma designs
- User interface choices
- Component structure

**Ruhan (Team Lead):**

- Overall architecture
- Tech stack justification
- Timeline and scope management
- Integration between frontend/backend

**Everyone should know:**

- Why three-tier architecture
- Why token authentication
- Why our specific database schema
- How security works

---

# FINAL NOTE

The professor will likely focus on:

1. **Database design** (most complex part)
2. **Security** (critical for healthcare)
3. **Scope decisions** (what's in/out and why)

Be confident but honest. If you don't know something, say "That's a good question. We focused on X because of Y, but we'd be happy to research Z for the next phase."

Good luck! 🎯